Politecnico di Torino
Degree course in Cybersecurity Engineering

Project 1 — Precise Scheduler for FreeRTOS

Operating Systems for Embedded Systems

Authors

Stefano Falcetta (s348753)
Cristian Sapia (s349451)
Stefano Calandrella (s362143)
Mattia Rabezzana (s358702)

Academic Year 2025/2026

February 18, 2026

Contents

1 Introduction **3**

2 System Architecture **5**

2.1 High-Level Architecture 5

2.2 Subframe-Based Organization 6

3 Implementation Choices and Rationale **7**

3.1 Separate APIs for HRT and SRT Tasks 7

3.2 Distinct Data Structures for HRT and SRT Tasks 7

4 Technical Manual - User-Level APIs **9**

4.1 Task Creation APIs 9

4.2 Common Parameters 10

4.3 User-Provided Stack Memory 10

4.3.1 Kernel-Provided Stack Memory Configuration 11

4.4 Scheduler Configuration Macros 11

4.4.1 FreeRTOS Native Macros 12

4.4.2 Custom Scheduler Configuration Macros 13

4.4.3 Include and Debug Macros 14

4.5 Tracing and Debugging 15

4.5.1 Trace Configuration Header 15

4.5.2 Trace Macro Definitions 15

4.5.3 Debugging and Trace Workflow 17

4.5.4 Usage Recommendations 18

5 Internal Scheduler Mechanics – Backend **19**

5.1 Backend Data Structures 19

5.2 Task Descriptors 20

5.3 Linked Lists for Task Management 20

5.4 Stack Management and Sharing 21

5.5 Stack Reinitialization 21

5.6 Scheduler Integration 22

5.7 FreeRTOS Integration 22
5.8 Workflow Diagrams 23
5.8.1 Low-Level Creation Wrapper 23
5.8.2 Stack Initialization 24
5.8.3 Hard Real-Time (HRT) Workflow 24
5.8.4 Soft Real-Time (SRT) Workflow 25
5.9 Validation 25

6 Runtime Logic **26**
6.1 Scheduler Startup 26
6.2 Tick Handling 26
6.3 Context Switching and Stack Initialization 27

7 Conclusion **28**

Chapter 1

Introduction

This document presents the design and implementation of a Precise Scheduler for FreeRTOS, developed as part of the *Operating Systems for Embedded Systems* course. The objective of the project is to extend the FreeRTOS kernel with deterministic, time-aware scheduling capabilities suitable for real-time embedded systems.
The scheduler introduces explicit support for two distinct task categories:

- **Hard Real-Time Tasks (HRT):** tasks characterized by strict temporal constraints defined by a start time and an end time.
- **Soft Real-Time Tasks (SRT):** tasks without hard deadlines, executed opportunistically within a predefined temporal window.

The scheduling model is based on a cyclic major frame / subframe organization. Each major frame is divided into a configurable number of subframes, and tasks are statically assigned to a specific subframe. Task assignment is performed *before the scheduler is started*, through custom task creation functions, and remains unchanged at runtime. All tasks are created using static allocation and are fully configured prior to the invocation of `vTaskStartScheduler()`.

The execution pattern repeats deterministically at each major frame iteration. In this context, determinism means that:

- the set of tasks executed in each subframe is fixed at configuration time;
- the execution order of HRT tasks within a subframe is fully determined by their start times;
- SRT tasks are executed in the order in which they are created within each subframe, using the remaining time budget of the subframe;
- no dynamic task creation, deletion, or reordering occurs during runtime.

As a result, the same temporal behavior is reproduced at every major frame cycle, assuming identical input conditions. The system is designed to integrate tightly with FreeRTOS, reusing its internal mechanisms where possible and extending them only where necessary to ensure predictability, temporal isolation, and low runtime overhead. In particular, the scheduler leverages the native FreeRTOS tick mechanism and context switching logic, while introducing a custom backend for timeline-driven scheduling and task activation.

Chapter 2

System Architecture

2.1 High-Level Architecture

The proposed scheduler extends the FreeRTOS kernel by introducing a precise and deterministic execution model based on two distinct categories of tasks: Hard Real-Time (HRT) tasks and Soft Real-Time (SRT) tasks.

The overall architecture is composed of the following main components:

- Custom user APIs for HRT and SRT task creation
- A configuration layer based on FreeRTOS-style macros
- Backend data structures for task organization
- Runtime scheduling logic integrated into the FreeRTOS kernel

HRT and SRT schedulers are implemented as independent and optional features. Each scheduler can be enabled or disabled at compile time through dedicated configuration macros. When both schedulers are enabled, HRT tasks are always assigned a higher priority than SRT tasks, ensuring correct real-time behavior and temporal isolation.

All tasks managed by the custom scheduler are internally created as *static FreeRTOS tasks*. No dynamic memory allocation is performed at runtime. Task Control Blocks (TCBs) are embedded directly inside the task descriptors, guaranteeing predictable memory usage and eliminating heap-related nondeterminism.

The backend implementation interacts directly with internal FreeRTOS kernel primitives rather than relying on the public task creation API. This allows tasks to be created without being automatically inserted into the ready list, preserving full control over activation timing.

Runtime scheduling logic is integrated at well-defined kernel execution points, namely:

- Scheduler startup (`vTaskStartScheduler`)

- System tick handling (`xTaskIncrementTick`)
- Context switching via PendSV (`vTaskSwitchContext`)

This design enables seamless coexistence with the native FreeRTOS scheduler while maintaining deterministic execution guarantees.

2.2 Subframe-Based Organization

The temporal execution model is organized using a subframe-based structure.

A *major frame* represents a complete scheduling cycle. Each major frame is divided into `configSUBFRAME_COUNT_PER_MAJOR` subframes. Each subframe contains its own set of HRT and SRT tasks.

Subframes are represented by a statically allocated array indexed by subframe identifier. This ensures constant-time access to scheduling data structures and avoids any runtime allocation.

HRT tasks within each subframe are inserted in sorted order according to their start time. This guarantees that, at runtime, HRT tasks can be executed sequentially without requiring additional sorting or searching operations.

SRT tasks within a subframe are organized in a singly linked list that preserves their creation order. As a consequence, SRT tasks are executed sequentially in the same order in which they are created, reflecting their soft real-time nature and keeping the scheduling logic simple and predictable.

This organization provides:

- Temporal isolation between task groups
- Predictable and bounded scheduling overhead
- A natural cyclic execution model

Chapter 3

Implementation Choices and Rationale

3.1 Separate APIs for HRT and SRT Tasks

Two distinct task creation APIs are provided: one for HRT tasks and one for SRT tasks. Each API closely follows the design style of FreeRTOS, including the availability of both static and dynamic creation variants.

The separation of APIs enforces a clear semantic distinction between task categories and enables category-specific validation logic.

Motivation:

- *Type safety*: it is impossible to create an HRT task without specifying temporal constraints such as start and end times.
- *API clarity*: the function name explicitly communicates the task category.
- *Specific validation*: HRT tasks are subject to overlap and deadline checks, while SRT tasks are validated to preserve their creation order within the subframe.
- *Self-documenting code*: improves readability and long-term maintainability.

Both static and dynamic API variants are supported. In static variants, the user provides the memory required to store the task descriptor.

All user inputs are validated at task creation time. Invalid parameters (e.g., out-of-range subframe identifiers or overlapping HRT intervals) are rejected immediately, preventing the construction of unschedulable configurations.

3.2 Distinct Data Structures for HRT and SRT Tasks

HRT and SRT tasks are represented using distinct data structures. This design choice avoids unnecessary complexity and reflects the fundamentally different scheduling requirements of the two task categories.

Each task descriptor embeds its own FreeRTOS TCB, eliminating the need for dynamic memory allocation or external memory provisioning for kernel structures.

HRT tasks include additional temporal metadata such as start and end times, while SRT tasks only store execution parameters. Both task types are linked together using singly linked lists.

Motivation:

- *Separation of responsibilities*: HRT tasks require strict temporal guarantees, while SRT tasks do not.
- *Compile-time safety*: misuse of task categories is prevented by the type system.
- *Simplified scheduling logic*: each scheduler operates on a minimal and purpose-specific data structure.

Singly linked lists are used instead of the doubly linked list abstraction provided by FreeRTOS. Backward traversal is never required, making the FreeRTOS list structure unnecessarily complex for this use case.

All HRT- and SRT-related data structures are conditionally compiled based on configuration macros. When a scheduler feature is disabled, no memory or runtime overhead is introduced.

Chapter 4

Technical Manual - User-Level APIs

4.1 Task Creation APIs

The scheduler exposes a set of user-level APIs for creating Hard Real-Time (HRT) and Soft Real-Time (SRT) tasks. The APIs are designed to closely follow FreeRTOS conventions while enforcing category-specific constraints at creation time.

Function	Type	Description
xTaskHrtCreate	HRT	Creates a hard real-time task; requires start and end times
xTaskHrtCreateStatic	HRT	Static variant; task control block provided by the user
xTaskSrtCreate	SRT	Creates a soft real-time task executed in leftover time
xTaskSrtCreateStatic	SRT	Static variant with user-provided task buffer

Table 4.1: Overview of the Task Creation APIs

The availability of static and dynamic variants is controlled by the standard FreeRTOS configuration macros `configSUPPORT_STATIC_ALLOCATION` and `configSUPPORT_DYNAMIC_ALLOCATION`. Static creation functions return `NULL` on failure, preserving consistency with FreeRTOS API semantics.
The full function signatures are reported below.

```
BaseType_t xTaskHrtCreate (
    TaskFunction_t pxTaskCode,
    const char * const pcName,
    void * pvParameters,
    uint32_t ulSubframeId,
    uint32_t ulStartTime,
    uint32_t ulEndTime,
    HRTHandle_t * pxTaskReturn
)

HRTHandle_t xTaskHrtCreateStatic (
    TaskFunction_t pxTaskCode,
    const char * const pcName,
```

```
    void * pvParameters,
    uint32_t ulSubframeId,
    uint32_t ulStartTime,
    uint32_t ulEndTime,
    HRTTask_t * const pxTaskBuffer
)

BaseType_t xTaskSrtCreate (
    TaskFunction_t pxTaskCode,
    const char * const pcName,
    void * pvParameters,
    uint32_t ulSubframeId,
    SRTHandle_t * pxTaskReturn
)

SRTHandle_t xTaskSrtCreateStatic (
    TaskFunction_t pxTaskCode,
    const char * const pcName,
    void * pvParameters,
    uint32_t ulSubframeId,
    SRTTask_t * const pxTaskBuffer
)
```

4.2 Common Parameters

All task creation functions share a set of common parameters that define the logical behavior of the task:

- **pxTaskCode**: Pointer to the function executed by the task. The function must conform to the standard FreeRTOS task signature.
- **pvParameters**: Pointer to user-defined data passed to the task at execution time. This parameter may be set to `NULL` if unused.
- **ulSubframeId**: Identifier of the subframe in which the task is scheduled. The value must be in the range `[0, configSUBFRAME_COUNT_PER_MAJOR - 1]`.

For HRT tasks, the parameters `ulStartTime` and `ulEndTime` define the execution window within the selected subframe. These constraints are validated at creation time to prevent temporal overlap and invalid scheduling configurations.

4.3 User-Provided Stack Memory

The scheduler relies on user-defined hooks to obtain stack memory for task execution:

```c
void vApplicationGetHrtStackMemory(
    StackType_t **ppxStackBuffer,
    uint32_t *pulStackSize
);

void vApplicationGetSrtStackMemory(
    StackType_t **ppxStackBuffer,
    uint32_t *pulStackSize
);
```

These functions are invoked during scheduler startup to retrieve the memory regions used as shared stacks for HRT and SRT tasks.
By delegating stack allocation to the application, the design avoids dynamic memory allocation at runtime and enables safe stack sharing. This approach is inspired by the `vApplicationGetIdleTaskMemory()` mechanism provided by FreeRTOS, while extending it to support deterministic real-time scheduling.

4.3.1 Kernel-Provided Stack Memory Configuration

The scheduler supports a configurable stack allocation mechanism controlled by the `configKERNEL_PROVIDED_STACK_MEMORY` macro.

When this macro is enabled, the scheduler provides default implementations of `vApplicationGetHrtStackMemory()` and `vApplicationGetSrtStackMemory()`, automatically allocating the memory required for the shared HRT and SRT stacks. In this configuration, stack memory is reserved by the kernel in a generic manner, without enforcing any specific memory placement strategy. This simplifies integration and allows the system to operate without requiring additional user configuration.

When `configKERNEL_PROVIDED_STACK_MEMORY` is set to 0, the responsibility for defining `vApplicationGetHrtStackMemory()` and `vApplicationGetSrtStackMemory()` is delegated to the user. This design choice enables full control over stack placement. By providing custom implementations of these functions, the user can place the shared stacks in specific memory regions according to application requirements.

This dual approach offers both ease of use and architectural flexibility: a ready-to-use default configuration for standard deployments, and a fully customizable memory placement strategy for advanced embedded scenarios.

4.4 Scheduler Configuration Macros

This section describes the configuration macros used to enable, customize, and tune the timeline-driven scheduler implemented in this project. These macros follow the FreeRTOS configuration style and allow the scheduler behavior to be fully determined at compile

time.

The configuration layer serves multiple purposes:

- enabling or disabling entire scheduler subsystems (HRT and SRT),
- defining the temporal structure of the timeline (major frame and subframes),
- selecting memory allocation strategies,
- configuring stack sizes and priority levels,
- optionally enabling debugging and tracing utilities.

Most macros are defined in profile-specific configuration headers (e.g., within `FreeRTOS Config.h` or project-specific `config/` files) and are intended to be adjusted per application profile, depending on timing constraints and available resources.

4.4.1 FreeRTOS Native Macros

The following macros are provided by FreeRTOS and directly influence task allocation mechanisms and kernel behavior. The scheduler implementation relies on these macros to determine which task creation variants are available and how memory is managed.

- `configSUPPORT_STATIC_ALLOCATION`
  Enable static task allocation support (standard FreeRTOS configuration).
  Possible values: `1` / `0`.
  If enabled, the static task creation APIs can be used to avoid dynamic heap usage.
- `configSUPPORT_DYNAMIC_ALLOCATION`
  Enable dynamic task allocation support (standard FreeRTOS configuration).
  Possible values: `1` / `0`.
  If enabled, dynamic task creation APIs are available.
- `configKERNEL_PROVIDED_STATIC_MEMORY`
  When enabled, the kernel supplies application static memory buffers.
  Possible values: `1` / `0`.
  If enabled, the kernel internally defines static stack memory for HRT/SRT tasks. It requires the user to define `configHRT_TASKS_STACK_SIZE` and `configSRT_TASKS _STACK_SIZE` if the corresponding HRT/SRT functionalities are enabled.

These macros are not redefined by the project but are leveraged to ensure full compatibility with standard FreeRTOS allocation semantics.

4.4.2 Custom Scheduler Configuration Macros

The following macros are introduced by the project and define the behavior of the timeline-driven scheduler. They control which scheduling features are compiled, how tasks are temporally organized, and how resources are assigned to each task class.

- `configENABLE_HRT_SCHEDULER`
  Enable Hard Real-Time (HRT) scheduler support.
  Possible values: `1` / `0`.
  When enabled, HRT task creation APIs are available. It can be used independently of the SRT scheduler.

- `configENABLE_SRT_SCHEDULER`
  Enable Soft Real-Time (SRT) scheduler support.
  Possible values: `1` / `0`.
  When enabled, SRT task creation APIs are available. It can be used independently of the HRT scheduler.

- `configSUBFRAME_COUNT_PER_MAJOR`
  Number of subframes per major frame.
  Typical values: `4`, `10`, `20`, etc.
  Combined with `configSUBFRAME_DURATION` to define the overall major frame duration.

- `configSUBFRAME_DURATION`
  Duration of each subframe (in milliseconds).
  Typical values: `25`, `100`, etc.
  The major frame duration is computed as:
  `configSUBFRAME_COUNT_PER_MAJOR * configSUBFRAME_DURATION`.
  This value is defined at compile-time in milliseconds.

- `configHRT_TASKS_STACK_SIZE`
  Stack size for HRT tasks (words or bytes depending on the target port).
  Typical value: `configMINIMAL_STACK_SIZE * 2`.
  Defines the stack memory size shared by all HRT tasks.

- `configSRT_TASKS_STACK_SIZE`
  Stack size for SRT tasks (words or bytes depending on the target port).
  Typical value: `configMINIMAL_STACK_SIZE`.
  Defines the stack memory size shared by all SRT tasks.

- `configHRT_TASKS_PRIORITY`
  Priority internally assigned by the scheduler to all HRT tasks.

Typical value: `configMAX_PRIORITIES - 1`.
HRT tasks should have a higher priority than SRT tasks.

- `configSRT_TASKS_PRIORITY`
  Priority internally assigned by the scheduler to all SRT tasks.
  Typical value: `tskIDLE_PRIORITY + 1`.
  SRT tasks should have a lower priority than HRT tasks.

The separation between HRT and SRT configuration parameters allows the two schedulers to be enabled independently and tuned according to different timing and resource requirements.

4.4.3 Include and Debug Macros

The scheduler optionally provides helper functions for inspecting and debugging the configured timeline. These macros control the inclusion of such utilities at compile time and introduce no overhead when disabled.

- `INCLUDE_vPrintSubframes`
  Enable printing of configured subframes in a human-readable format.
  Possible values: `1` / `0`.
  Debugging flag, not intended for production builds.

- `INCLUDE_vPrintSubframesJson`
  Enable printing of configured subframes in JSON format.
  Possible values: `1` / `0`.
  Useful for automated parsing during testing, relies on unsafe escaping and therefore not intended for production builds.

Summary

- **Enable flags** (`configENABLE_HRT_SCHEDULER`, `configENABLE_SRT_SCHEDULER`) allow independent activation of HRT and SRT subsystems.

- **Timeline parameters** (`configSUBFRAME_COUNT_PER_MAJOR`, `configSUBFRAME_DURATION`) define the temporal structure of the scheduler (e.g., 4 subframes $\times$ 25 ms = 100 ms major frame).

- **Allocation macros** determine whether static and/or dynamic task creation APIs are available.

- **Stack sizes and priorities** define resource limits and enforce correct preemption between HRT and SRT tasks.

- **Debug macros** provide optional visibility into scheduler state without affecting runtime performance when disabled.

4.5 Tracing and Debugging

4.5.1 Trace Configuration Header

The file `tracingConfig.h` defines user-level macros to instrument runtime events in the timeline-driven scheduler. These macros provide a lightweight yet precise mechanism to capture both Hard Real-Time (HRT) and Soft Real-Time (SRT) task behavior, as well as system tick activity, producing structured, human-readable output for debugging, verification, and performance analysis.

When `task.c` is compiled and this header is included, the trace macros are automatically configured and enabled, ensuring that runtime events are emitted in the structured format expected by the accompanying Python analysis tools. This guarantees that the generated logs can be directly parsed for offline processing, trace inspection, and Gantt chart visualization without requiring additional instrumentation.

Purpose

- Capture task switch-in and switch-out events (`traceTASK_SWITCHED_IN`, `traceTASK_SWITCHED_OUT`), enabling identification of task preemption and execution intervals.
- Log HRT deadline misses (`traceTASK_HRT_DEADLINE_MISS`) and SRT overflows (`traceTASK_SRT_OVERFLOW`), providing real-time feedback on scheduling violations.
- Record task completion events (`traceTASK_HRT_COMPLETE`, `traceTASK_SRT_COMPLETE`)
  for offline validation of task execution against expected temporal profiles.
- Track system tick entry and exit (`traceENTER_xTaskIncrementTick`, `traceRETURN_xTaskIncrementTick`) to accurately measure scheduler overhead and ISR impact.
- Dump subframe configuration at scheduler startup (`traceENTER_vTaskStartScheduler`), allowing verification of the mapping of tasks to subframes.

4.5.2 Trace Macro Definitions

Macros provided by FreeRTOS:

- `traceTASK_SWITCHED_IN()`
  Logs task switch-in events: `printf("[TRACE] %08llu TASK %s n", ullGetCpuTicks(), pxCurrentTCB->pcTaskName);`

- `traceTASK_SWITCHED_OUT()`
  Logs task switch-out events: `printf("[TRACE] %08llu TASK_END %s n", ullGetCpuTicks(), pxCurrentTCB->pcTaskName);`

- `traceENTER_xTaskIncrementTick()`
  Logs system tick entry: `printf("[TRACE] %08llu SysTick n", ullGetCpuTicks());`

- `traceRETURN_xTaskIncrementTick(_)`
  Logs system tick exit: `printf("[TRACE] %08llu SysTick_END n", ullGetCpuTicks());`

Project-specific trace macros:

- `traceENTER_vTaskStartScheduler()`
  Dumps subframe configuration at scheduler startup: `printf("[TASKS] %lu", configSUBFRAME_DURATION); vPrintSubframesJson();`

- `traceTASK_HRT_DEADLINE_MISS(pxTask)`
  Logs HRT task deadline misses: `printf("[TRACE] %08llu MISS %s n", ullGetCpuTicks(), pxTask->pcTaskName);`

- `traceTASK_SRT_OVERFLOW(pxTask)`
  Logs SRT task overflows: `printf("[TRACE] %08llu OVERFLOW %s n", ullGetCpuTicks(), pxTask->pcTaskName);`

- `traceTASK_HRT_COMPLETE(pxTask)`
  Logs HRT task completion: `printf("[TRACE] %08llu COMPLETE %s n", ullGetCpuTicks(), pxTask->pcTaskName);`

- `traceTASK_SRT_COMPLETE(pxTask)`
  Logs SRT task completion: `printf("[TRACE] %08llu SRT_COMPLETE %s n", ullGetCpuTicks(), pxTask->pcTaskName);`

Explanation

- **Timestamping:** Each macro uses `ullGetCpuTicks()` to produce high-resolution timestamps, ensuring that captured events accurately reflect the execution timing of tasks and interrupts.

- **Task Events:** Switch-in and switch-out macros allow precise monitoring of when a task gains or loses the CPU. Completion and deadline miss macros provide direct insight into whether tasks respect their temporal constraints.
- **System Tick Tracing:** The entry and exit of the system tick ISR are traced, enabling detailed analysis of scheduler overhead and interrupt impact on task execution.
- **Integration with FreeRTOS:** These macros are called from standard FreeRTOS hooks and timeline-driven scheduler logic, maintaining full compatibility with the RTOS kernel and minimizing intrusion.
- **SRT/HRT Unification:** SRT task events are aliased to HRT macros for trace purposes, preserving semantic distinction while simplifying logging and analysis.
- **Subframe Configuration Dumping:** `vPrintSubframesJson()` prints structured JSON-like subframe information at scheduler startup. This enables automated tools and scripts to parse task allocation, supporting trace-driven verification.

4.5.3 Debugging and Trace Workflow

The trace macros are consumed by Python-based tools for asynchronous capture, analysis, and visualization of scheduler behavior:

- **Capture:** The `TraceMonitor` class in `tracing.py` reads runtime traces asynchronously from a QEMU-emulated target, collecting events such as HRT/SRT task starts, completions, deadline misses, and system tick entries.
- **Event Processing:** Each captured trace line is parsed to determine the task type, start/end timestamps, and any missed deadlines. Task intervals are stored in structured tuples for further computation.
- **Analysis:** The monitor computes metrics such as total task execution time, interrupt handling time, scheduler overhead, and verifies adherence to subframe boundaries and HRT/SRT temporal constraints.
- **Visualization:** Using the `plot_gantt` function in `graph.py`, the captured traces are plotted as Gantt charts, showing HRT and SRT tasks, system ticks, and any missed deadlines, enabling intuitive visual inspection of the scheduler's temporal behavior.
- **Trace Verification:** Automated checks ensure that:
  1. Each HRT task executes exactly once per subframe.

2. SRT tasks run in the correct subframe and in the expected order.
3. Scheduler overhead does not exceed predefined thresholds.

- **Interactive Debugging:** Users can optionally view the Gantt charts for failing tests or temporal anomalies, providing immediate feedback on task scheduling correctness.

4.5.4 Usage Recommendations

1. Enable tracing only in debug builds to minimize runtime overhead in production environments.
2. Combine macro traces with subframe dumps for structured offline analysis and automated verification.
3. Use Python scripts (`debug.py`, `tracing.py`, `graph.py`) to capture, parse, and visualize traces, facilitating detailed temporal analysis and debugging of both HRT and SRT tasks.
4. Cross-validate traces with expected subframe allocation and major frame limits to detect scheduling violations, missed deadlines, or excessive scheduler overhead.

By defining these macros and integrating them with asynchronous capture and Gantt visualization tools, developers can perform comprehensive runtime verification of the timeline-driven scheduler, ensuring correct task preemption, adherence to deadlines, and accurate measurement of system overhead while maintaining minimal impact on the RTOS kernel.

Chapter 5

Internal Scheduler Mechanics – Backend

5.1 Backend Data Structures

The backend scheduling logic is based on a statically allocated array of subframes, representing the internal structure of a major frame.

```
typedef struct xSUBFRAME
{
#if configENABLE_HRT_SCHEDULER
    HRTTask_t * pxFirstHrtTask;
    uint32_t ulHrtCount;
#endif

#if configENABLE_SRT_SCHEDULER
    SRTTask_t * pxFirstSrtTask;
    SRTTask_t * pxLastSrtTask;
    uint32_t ulSrtCount;
#endif
} Subframe_t;

static Subframe_t axMajorSubframes[ configSUBFRAME_COUNT_PER_MAJOR ];
```

Each element of the `axMajorSubframes` array represents a subframe and contains independent scheduling data for HRT and SRT tasks. The array size is fixed at compile time through the `configSUBFRAME_COUNT_PER_MAJOR` macro, guaranteeing constant memory usage.

Conditional compilation is used to include only the data structures required by the enabled scheduler features. When either the HRT or SRT scheduler is disabled, the corresponding fields are completely removed, introducing no memory overhead.

Each subframe maintains:

- A pointer to the first HRT task and a task count
- A pointer to the first and last SRT task and a task count

This organization enables constant-time access to scheduling data and ensures complete temporal isolation between subframes.

5.2 Task Descriptors

HRT and SRT tasks are represented using dedicated task descriptor structures.

```
typedef struct xHRT_TASK
{
    StaticTask_t xTCB;
    TaskFunction_t pxTaskCode;
    void * pvParameters;
    uint32_t ulStartTime;
    uint32_t ulEndTime;
    struct xHRT_TASK * pxNext;
}HRTTask_t;

typedef struct xSRT_TASK
{
    StaticTask_t xTCB;
    TaskFunction_t pxTaskCode;
    void * pvParameters;
    struct xSRT_TASK * pxNext;
}SRTTask_t;
```

Each task descriptor embeds a `StaticTask_t` structure, which corresponds to the FreeRTOS Task Control Block (TCB). This allows tasks to be created using the FreeRTOS static task mechanism while retaining full control over task lifecycle management.

HRT task descriptors include explicit temporal constraints, namely start and end times, which are expressed relative to the subframe execution. SRT task descriptors do not include temporal metadata, reflecting their best-effort execution semantics.

Both task types are linked together using singly linked lists. Backward traversal is never required, making this representation optimal in terms of memory footprint and runtime complexity.

5.3 Linked Lists for Task Management

HRT tasks within a subframe are stored in a singly linked list ordered by increasing start time. This ordering is enforced at task creation time, ensuring that the list is always ready for sequential execution during runtime.

As a result, HRT scheduling requires only a linear traversal of the list, without any need for sorting or searching operations during execution.

SRT tasks are stored in a singly linked list. Insertion is performed in constant time by maintaining both head and tail pointers. This design reflects the soft real-time nature of SRT tasks and guarantees predictable and minimal insertion overhead.

This linked-list-based approach ensures:

- Deterministic traversal time
- Minimal memory overhead
- Direct alignment with the scheduling semantics of each task category

5.4 Stack Management and Sharing

The scheduler employs a shared-stack execution model to minimize memory usage while preserving deterministic behavior. Stack sharing is possible because the scheduler is designed for a **single CPU** environment.

Two shared stacks are statically allocated:

- One shared stack for all HRT tasks
- One shared stack for all SRT tasks

HRT tasks are strictly non-preemptive with respect to each other. At any point in time, only one HRT task can be executing, making stack sharing safe and eliminating the need for per-task stacks.

SRT tasks, on the other hand, may be preempted by HRT tasks. For this reason, a separate shared stack is used for SRT tasks to preserve execution context across preemptions.

This design significantly reduces memory consumption while maintaining correctness, temporal predictability, and safe execution within a single-core system.

5.5 Stack Reinitialization

The scheduler employs explicit reinitialization of the shared stacks used by HRT and SRT tasks. Rather than performing this operation as a standalone step at the end of a major frame, stack reinitialization is carried out *just before the execution of a new task*, inside the PendSV context switch handler.

Whenever the scheduler selects a new task to run, the corresponding shared stack is reinitialized by resetting the stack pointer to its initial state through `prvResetTaskStack()`,

which internally invokes `pxPortInitialiseStack()`. This ensures that each task starts execution with a clean stack context.

Stack reinitialization resets the stack pointer to its initial value, guaranteeing that no residual execution state from previously executed tasks or earlier major frames is carried over.

This mechanism provides:

- **Deterministic execution across cycles**: each task execution begins from a known and reproducible stack state.
- **Isolation between tasks**: shared stacks can be safely reused without interference from prior executions.
- **Protection against stack corruption accumulation**: stale stack contents are discarded before each task activation.
- **Minimal runtime overhead**: stack initialization is performed only when a task is actually dispatched, and always within the existing PendSV context switch.

5.6 Scheduler Integration

The custom scheduling logic is integrated directly into the FreeRTOS kernel by extending the `xTaskIncrementTick()` function.

Embedding the scheduler at this point provides several advantages:

- Maximum temporal resolution, as decisions are made at every system tick
- Seamless integration with the native FreeRTOS scheduling mechanism
- Constant-time scheduling decisions based on pre-organized data structures

By leveraging the existing kernel execution flow, the custom scheduler avoids duplicating scheduling logic and maintains full compatibility with standard FreeRTOS behavior.

5.7 FreeRTOS Integration

The custom scheduling logic is fully integrated into the FreeRTOS kernel without modifying its public API. The integration leverages FreeRTOS internal mechanisms while preserving the native multilevel priority queue system, ensuring that tasks created outside the timeline-driven scheduler can still coexist correctly with HRT and SRT tasks.

The integration relies on two core kernel functions:

1. **xTaskIncrementTick()**: Executed at every system tick, this function computes the current subframe, activates tasks at their specified start times, checks deadline violations for HRT tasks, and manages SRT tasks that did not complete within the previous subframe. All scheduling decisions are performed in a deterministic and constant-time manner.

2. **vTaskSwitchContext()** (PendSV): Responsible for context switching. Task stacks are initialized on-demand through `pxPortInitialiseStack()` when a task is selected for execution, ensuring correct context restoration and minimal initialization overhead. By leveraging FreeRTOS's existing ready list structure and multilevel priority queue, HRT tasks always preempt SRT tasks as defined by the scheduler, while preserving proper prioritization for all other tasks in the system.

This design guarantees deterministic timeline-driven scheduling, full compatibility with FreeRTOS internal mechanisms, and seamless coexistence with tasks managed by the native multilevel priority queue.

5.8 Workflow Diagrams

This section describes the internal execution flow of the scheduler by analyzing the main creation and runtime workflows. Each diagram highlights how the custom scheduler extends the FreeRTOS kernel while preserving its internal invariants and execution model.

5.8.1 Low-Level Creation Wrapper

```
prvCreateStaticNoAddToReady (wrapper)
  |
  +--> prvInitialiseNewTask (FreeRTOS)
```

The function `prvCreateStaticNoAddToReady()` acts as a low-level wrapper around the internal FreeRTOS routine `prvInitialiseNewTask()`. Unlike the standard task creation path, this wrapper deliberately avoids inserting the newly created task into the FreeRTOS ready list.
In vanilla FreeRTOS, task creation immediately makes the task eligible for scheduling. This behavior is incompatible with the subframe-based temporal model, where tasks must only become runnable at specific, precomputed time instants.
By decoupling *task creation* from *task activation*, the scheduler preserves full control over when a task enters the ready state, enabling deterministic and time-driven execution without relying on task suspension or deletion.

5.8.2 Stack Initialization

```
prvResetTaskStack
  |
  +--> pxPortInitialiseStack (FreeRTOS)
```

The function `prvResetTaskStack()` explicitly reinitializes a task stack by invoking the architecture-specific FreeRTOS routine `pxPortInitialiseStack()`.
In the standard FreeRTOS execution model, stack initialization occurs once at task creation time. In contrast, the proposed scheduler reinitializes the stack every time a task is (re)activated within a subframe or at the beginning of a new major frame.
This design choice is essential to support stack sharing and to guarantee deterministic execution across repeated major frames. By resetting the stack, all residual execution state from previous activations is eliminated, ensuring identical behavior across cycles.

5.8.3 Hard Real-Time (HRT) Workflow

```
xTaskHrtCreate / Static (USER API)
  |
  +--> prvCreateHrtTask        (TRUE LOGIC)
        |
        +--> prvInitialiseNewHrtTask
              |
              +--> prvCreateStaticNoAddToReady
        |
        +--> Update HRT subframe list
```

The HRT creation workflow begins at the user API level and is delegated to `prvCreateHrtTask()`, which implements the core logic of HRT task creation.
This function performs all HRT-specific validation, including:

- Subframe bounds checking
- Validation of start and end times
- Detection of temporal overlaps with existing HRT tasks

After successful validation, `prvInitialiseNewHrtTask()` initializes the task metadata and invokes `prvCreateStaticNoAddToReady()` to create the underlying FreeRTOS task without activating it.
Finally, the task is inserted into the subframe HRT list in start-time order. This guarantees runtime-efficient traversal and constant-time activation decisions during scheduling.

5.8.4 Soft Real-Time (SRT) Workflow

```
xTaskSrtCreate / Static (USER API)
  |
  +--> prvCreateSrtTask        (TRUE LOGIC)
        |
        +--> prvInitialiseNewSrtTask
              |
              +--> prvCreateStaticNoAddToReady
        |
        +--> Update SRT subframe list
```

The SRT creation workflow mirrors the structure of the HRT path but applies different semantic constraints.
The function `prvCreateSrtTask()` validates the subframe identifier and appends the task to the SRT list of the corresponding subframe, preserving the order in which tasks are created. No temporal constraints are enforced beyond subframe membership.
As with HRT tasks, the underlying FreeRTOS task is created but not immediately scheduled. This ensures that SRT tasks execute exclusively during the slack time left by HRT tasks within each subframe.

5.9 Validation

All task parameters are validated at creation time. Invalid configurations are rejected immediately, preventing the introduction of runtime checks and ensuring that scheduling decisions remain constant-time.
For dynamic APIs, the functions return `pdFALSE` on failure. For static APIs, `NULL` is returned, fully adhering to FreeRTOS conventions.

Chapter 6

Runtime Logic

This chapter describes the runtime behavior of the custom scheduler, detailing how tasks are initialized, activated, and managed during system execution. The scheduling logic is tightly integrated into the FreeRTOS kernel lifecycle and leverages existing kernel mechanisms to guarantee deterministic behavior.

6.1 Scheduler Startup

Scheduler initialization occurs during the execution of `vTaskStartScheduler()`.

At this stage, the kernel invokes user-defined memory provider functions to retrieve the stack regions assigned to HRT and SRT tasks. These functions supply pointers to statically allocated stack memory and their corresponding sizes, following a model inspired by the FreeRTOS idle task static memory provision.

Once the stack regions are obtained, the scheduler initializes the metadata of all tasks associated with the first subframe of the major frame. This includes:

- Resetting the shared stack pointers
- Initializing the FreeRTOS task control blocks (TCBs)
- Preparing tasks that must be ready for execution at scheduler start

Only tasks belonging to the initial subframe are added to the FreeRTOS ready lists. Tasks associated with subsequent subframes remain inactive until explicitly activated by the runtime scheduler logic.

6.2 Tick Handling

Runtime scheduling decisions are performed at each system tick through the `xTaskIncrementTick()` function.
At every tick, the scheduler executes the following operations:

- Determines the current subframe based on the elapsed time within the major frame
- Detects deadline overruns of active HRT tasks and enforces preemption if required
- Handles SRT tasks that did not complete execution within the previous subframe
- Activates HRT tasks whose start time matches the current tick
- Inserts the first SRT task of the new subframe into the ready list

This design ensures that all scheduling decisions are made with tick-level temporal precision, while avoiding dynamic memory allocation or complex runtime computations.

6.3 Context Switching and Stack Initialization

Context switching is performed by the FreeRTOS kernel through the `vTaskSwitchContext()` function, which is triggered by the PendSV exception.

The custom scheduler extends this phase by initializing task stacks only when a task is selected for execution. Specifically, when an HRT or SRT task is about to run, its stack is reinitialized using the `prvResetTaskStack()` routine, which internally relies on the `pxPortInitialiseStack()` port-layer function.

This deferred stack initialization strategy ensures:

- Correct restoration of execution context
- Safe stack sharing among tasks of the same category
- Minimal initialization overhead at runtime

When an SRT task terminates normally, the scheduler automatically activates the next SRT task registered in the subframe, preserving the original creation order and ensuring continuous best-effort execution until the subframe ends or the task list is exhausted.

Chapter 7

Conclusion

This work presented a custom real-time scheduler integrated into the FreeRTOS kernel, designed to support both Hard Real-Time (HRT) and Soft Real-Time (SRT) task models within a deterministic and configurable execution framework.

By introducing dedicated user APIs, distinct data structures, and a subframe-based temporal organization, the scheduler enforces strict temporal constraints for HRT tasks while preserving best-effort execution for SRT tasks. All scheduling decisions are performed at tick granularity and in constant time, ensuring predictable runtime behavior and bounded overhead.

The design deliberately avoids dynamic memory allocation at runtime, relying instead on static task creation, shared stack management, and explicit stack reinitialization to guarantee memory predictability and execution repeatability across major frames.

Overall, the implemented solution extends FreeRTOS without altering its core semantics, maintaining full kernel compatibility while enabling precise real-time scheduling. This approach makes the scheduler well suited for embedded systems that require deterministic behavior, low overhead, and clear separation between task categories.